Assessing Oracle Quality with Checked Coverage

David Schuler
Saarland University – Computer Science
Saarbrücken, Germany
ds@cs.uni-saarland.de

Andreas Zeller
Saarland University – Computer Science
Saarbrücken, Germany
zeller@cs.uni-saarland.de

***Abstract*—A known problem of traditional coverage metrics is that they do not assess *oracle quality*—that is, whether the computation result is actually checked against expectations. In this paper, we introduce the concept of *checked coverage*—the dynamic slice of covered statements that actually influence an oracle. Our experiments on seven open-source projects show that checked coverage is a sure indicator for oracle quality—and even more sensitive than mutation testing, its much more demanding alternative.**

***Keywords*-test suite quality, coverage metrics, dynamic slicing, mutation testing**

I. Introduction

How can I ensure that my program is well-tested? The most widespread metric to assess test quality is *coverage*. Test coverage measures the percentage of code features such as statements or branches that are executed during a test. The rationale is that the higher the coverage, the higher the chances of catching a code feature that causes a failure—a rationale that is easy to explain, but which relies on an important assumption. This assumption is that we actually are able to *detect* the failure. It does not suffice to cover the error, we also need a means to detect it.

As it comes to detecting arbitrary errors, it does not make a difference whether an error is detected by the test (e.g., a mismatch between actual and expected result), by the program itself (e.g., a failing assertion), or by the run-time system (e.g., a dereferenced null pointer). To validate computation results, though, we need checks in the test code and in the program code—checks henceforth summarized as *oracles*. A high coverage does not tell anything about oracle quality. It is perfectly possible to achieve a 100% coverage and still not have any result checked by even a single oracle. The fact that the run-time system did not discover any errors indicates robustness, but does not tell anything about functional properties.

As an example of such a mismatch between coverage and oracle quality, consider the test for the `PatternParser` class from the JAXEN XPATH library, shown in Figure 1. This test invokes the parser for a number of paths and achieves a statement coverage of 83% in the parser. However, none of the parsed results is actually checked for any property. The parser may return utter garbage, and this test would never notice it.

```
public void testValidPaths() throws
  JaxenException, SAXPathException {
  for (int i = 0; i < paths.length; i++) {
    String path = paths[i];
    Pattern p = PatternParser.parse(path);
  }
}
```

Figure 1. A test without outcome checks.

To assess oracle quality, a frequently proposed approach is *mutation testing*. Mutation testing seeds artificial errors (*mutants*) into the code and assesses whether the test suite finds them. A low score of detected mutants implies low coverage (i.e., the mutant was not executed) or low oracle quality (i.e., its effects were not checked). Unfortunately, mutation testing is costly; even with recent advancements, there still is manual work involved to weed out equivalent mutants.

In this paper, we introduce an alternative, cost-efficient way to assess oracle quality. Using dynamic slicing, we determine the *checked coverage*—statements that were not only executed, but that actually contribute to the results checked by oracles. In Figure 1, the checked coverage is 0%, because none of the results ever flows into a run-time check (again, in contrast to the 83% traditional coverage). However, adding a simple assertion that the result is non-null already increases the checked coverage to 65%; adding further assertions on the properties of the result further increases the checked coverage.

Using checked coverage as a metric rather than regular coverage brings significant advantages:

- Few or insufficient oracles immediately result in a low checked coverage, giving a more realistic assessment of test quality.
- Statements that are executed, but whose outcomes are never checked would be considered *uncovered.* As a consequence, one would improve the oracles to actually check these outcomes.
- Rather than focusing on executing as much code as possible, developers would focus on checking as many results as possible, catching more errors in the process.
- To compute checked coverage, one only needs to run

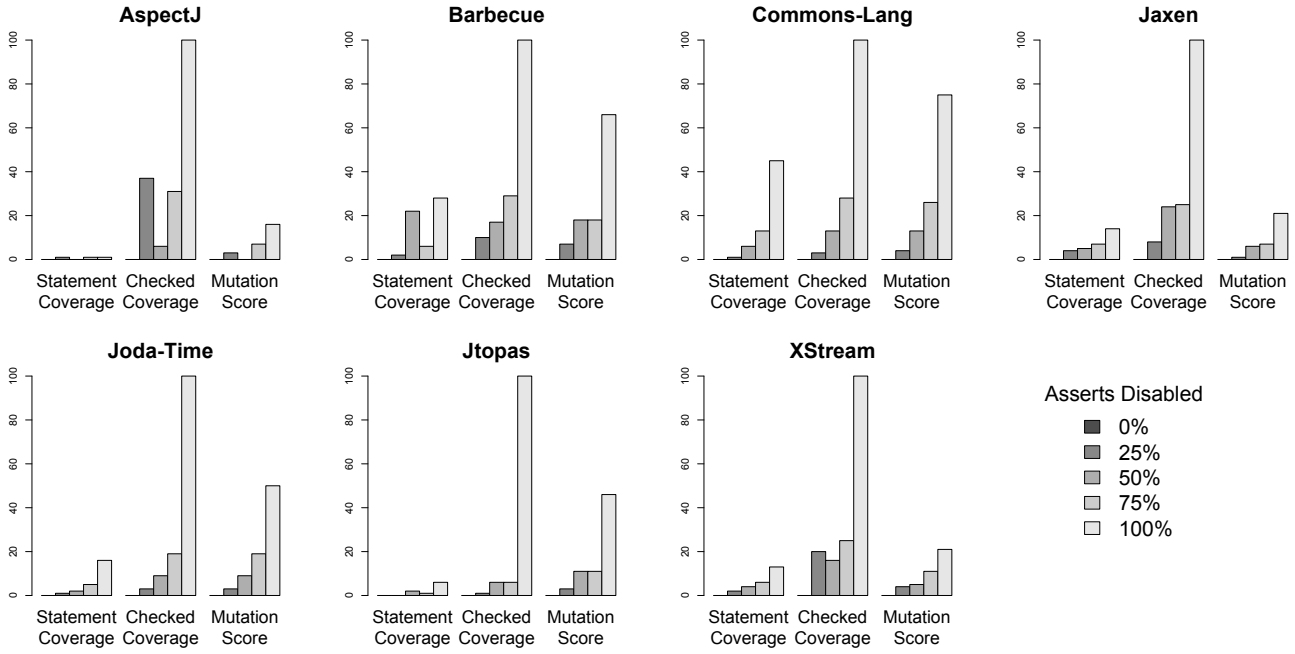


Figure 6. Decrease of the coverage values relative to the coverage values of the original test suite.

Checked coverage is more sensitive to missing assertions than statement coverage and mutation testing.

Note that when *all checks* are removed, checked coverage drops to 0% for all projects. This is by construction of the approach, as there are no statements left to slice from once all checks are removed from the test suite. Since we are interested in *poor* rather than nonexistent oracles, the results obtained for the decays of 25–75% are much more meaningful.

D. Explicit and Implicit Checks

As mutation testing is also a measure of a test suite's assertion quality, we might also expect that the mutation score dramatically drops when no checks are left. However, in the previous experiment, we have seen that test suites with no assertions still detect a significant fraction of the mutations (43 % on average). The reason for this is that a mutation can be detected by an *explicit* check of the test suite or by *implicit* checks of the runtime system. For modern object-oriented and restrictive programming languages like Java, many mutations are detected through theses implicit runtime checks.

JAVALANCHE, the mutation testing tool that we use for this study, uses the distinction made by JUnit in order to classify a mutation as either detected by explicit or implicit checks. JUnit classifies a test that does not pass either as a failure (explicit), when an `AssertionError` was thrown (e.g by an JUnit assertion method), or as an error (implicit), when any other exception was thrown. As we were interested how pronounced this effect is, we computed the fraction of mutations detected by assertions for the original test suite and the test suite with all checks removed.

Table III
MUTATIONS DETECTED BY EXPLICIT AND IMPLICIT CHECKS.

Project Name	Original Test Suite Total	Original Test Suite Explicit%	All checks removed Total	All checks removed Explicit%
ASPECTJ	5736	63%	5529	53%
BARBECUE	1036	59%	357	6%
JODA-TIME	12164	56%	6094	15%
JAXEN	4154	38%	3298	12%
JTOPAS	1295	57%	597	0%
COMMONS	13415	68%	3377	16%
XSTREAM	7764	35%	5156	5%
Total	**45564**	**55%**	**24408**	**21%**

Table III gives the results for each project. The first two values are for the original test suite. First, the total number of detected mutations (column 2) is given, and then the percentage of them that are detected by implicit checks (column 3). The remaining mutations are detected by explicit checks. The two last columns give the corresponding values for the test suite with all assert-statements removed.

For the original test suite, 32–65% of the mutations are detected by implicit checks. This is mainly due to `NullPointerExceptions` caused by mutations.

Almost half (45%) of the mutations are detected by implicit checks.

The test suites with all checks removed still detect 54% of the mutations that are detected by the original test suite; on average, 21% of the detected mutations are found by explicit